

SIMATS ENGINEERING
CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192511426
Name	K.B.Devarshan

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Code of Conduct:

*I_(K.B.Devarshan/192511426)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

ANSWERS:

1. AI-Powered Drones for Emergency Medicine Delivery (Flood-Affected Region)

1.1 Greedy Best-First Search (GBFS) Under Uncertainty

- **Behavior:** GBFS selects the node that appears closest to the goal by relying strictly on the heuristic function $h(n)$ (the straight-line Euclidean distance to the medical drop-off point). It ignores the cumulative cost of the path already traversed, $g(n)$.
- **Strengths:**
 - Extremely fast initial decision-making, which is critical in urgent medical scenarios.
 - Low memory overhead compared to exhaustive search methods.
- **Weaknesses:**
 - **Not optimal:** It can easily be seduced by a geographically direct route that is blocked by floodwaters or features high weather-related movement costs.
 - **Susceptible to dead ends:** If a straight-line path leads into an unpassable flooded valley, GBFS is forced into heavy backtracking, wasting critical drone battery life and increasing delivery delays.

1.2 A* Search: Balancing Cost and Heuristics

- **Behavior:** A* evaluates nodes using the evaluation function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual path cost incurred so far (incorporating real-time terrain difficulty, weather resistance, and distance), and $h(n)$ is the estimated remaining straight-line distance.

- **Balancing Mechanism:**
 - By summing $g(n)$ and $h(n)$, A* balances **exploitation** (moving toward the target via $h(n)$) with **exploration/safety** (avoiding costly or hazardous paths via $g(n)$).
 - If a sudden weather front increases flight resistance on a direct route, $g(n)$ spikes, prompting A* to explore a slightly longer but clearer and safer alternative path.

1.3 Impact of Heuristic Accuracy

- **Underestimating Heuristic (Admissible):** If $h(n) \leq h^*(n)$ (true cost), A* is guaranteed to find the mathematically optimal route. However, if $h(n)$ is too optimistic (e.g., ignoring major wind resistance), the drone may inspect many unnecessary intermediate nodes before finding the true best path.
- **Overestimating Heuristic (Inadmissible):** If $h(n) > h^*(n)$, A* loses its optimality guarantee. The search behaves more like GBFS—prioritizing speed over route efficiency—which can cause the drone to take a physically demanding route that drains its battery prematurely.

1.4 Dynamic Changes and Blocked Paths

- Static offline algorithms fail when roads/airspace become blocked mid-flight.
- When a path is blocked, standard A* must either invalidate its open/closed lists and replan from scratch, causing high computational latency, or risk flying into a hazard. Dynamic environments require incremental replanning where previous search trees are repaired rather than discarded.

1.5 Recommendation and Justification

- **Recommended Algorithm:** *D Lite (Dynamic A)* or *Anytime Repairing A (ARA)*.
- **Justification:**
 - **Real-Time Constraints:** D* Lite incrementally repairs the search tree when new obstacle data (flood blockages) is perceived by sensors, computing new paths in milliseconds rather than replanning from scratch.
 - **Safety & Optimality:** It maintains the optimal cost-balancing properties of A* while adapting instantly to real-time telemetry, ensuring the emergency medical payload arrives safely without battery exhaustion.

2. Smart City Traffic Signal Optimization

2.1 Optimization Formulation & Inefficiency of Traditional Search

- **Objective Function to Minimize:**

$$f(S) = w_1 \cdot T_{\text{wait}}(S) + w_2 \cdot F_{\text{consume}}(S) + w_3 \cdot C_{\text{congestion}}(S)$$

where S represents the vector of traffic signal cycle lengths, phase splits, and offsets across hundreds of intersections, and w_1, w_2, w_3 are priority weights.

- **Why Traditional Search is Inefficient:**
 - **Combinatorial Explosion:** For N intersections, each with k possible timing configurations, the search space size is k^N . Uninformed or simple informed search algorithms cannot traverse this space in real time.
 - **Continuous/Dynamic State Space:** Traffic is not static; arrival rates change continuously, making fixed-path search trees obsolete before a solution terminates.

2.2 Hill Climbing & Its Limitations

- **Mechanism:** Hill Climbing starts with a random or current signal timing configuration S_0 . It evaluates neighboring configurations (e.g., adding/subtracting 5 seconds to a green light phase) and moves to the neighbor that improves the objective function $f(S)$ the most.
- **Limitations:** It is a purely greedy local search; it never accepts a move that temporarily worsens traffic, causing it to get trapped easily.

2.3 Local Maxima, Plateaus, and Ridges (Real-World Interpretation)

Topological Issue	Definition in Optimization	Real-World Traffic Interpretation
Local Maxima / Minima	A state that is better than all immediate neighbors, but inferior to the global optimum.	A downtown corridor where adjusting one light makes local traffic slightly better, but prevents a city-wide green wave sequence.
Plateau	A flat area of the state space where neighboring states have identical evaluation scores.	Changing signal timings by ± 2 seconds during a severe gridlock yields no measurable change in queue lengths, leaving the algorithm directionless.
Ridge	A slope that is higher than surrounding areas but cannot be traversed by single-variable moves.	Two adjacent intersections require simultaneous timing adjustments to clear a bottleneck; changing either light independently worsens congestion.

2.4 Overcoming Limitations: Simulated Annealing & Genetic Algorithms

- **Simulated Annealing (SA):** Introduces a probability of accepting *worse* signal configurations governed by a temperature parameter T :

$$P(\text{accept}) = e^{-\frac{\Delta E}{T}}$$

Early in the process (high T), the system tests radical timing shifts to escape local minima. As T cools, it converges on a globally optimal traffic flow.

- **Genetic Algorithms (GAs):** Maintains a **population** of entire city timing schedules. Through **crossover** (combining successful corridor timings from two different

schedules) and **mutation** (randomly tweaking a signal phase), GAs explore multiple regions of the search space in parallel, easily navigating ridges and plateaus.

2.5 Recommendation and Justification

- **Recommended Approach: Genetic Algorithms (GA)** hybridized with real-time feedback.
- **Justification:**
 - **Scalability:** GAs evaluate candidates in parallel, making them capable of handling hundreds of interconnected intersections simultaneously.
 - **Adaptability:** By seeding the GA population with the *current* real-world traffic schedule alongside mutated variations, the system continuously evolves signal timings to match shifts in morning/evening rush-hour dynamics without resetting.

3. Autonomous Mars Rover Exploration

3.1 Need for an Online Search Agent

- **Offline Search** computes a complete trajectory from start to finish before taking a single physical action. This is impossible on Mars because the rover lacks a complete, high-resolution topographical map beforehand.
- **Online Search** interleaves computation and action: the rover senses its immediate vicinity, computes the next safest step, executes the physical movement, and observes the outcome.

3.2 Challenges of Partial Observability and Dynamic Environments

- **Partial Observability:** Onboard cameras and LiDAR only perceive terrain within a limited radius. Hidden drop-offs, soft sand traps, or sharp rocks behind dunes remain invisible until approached.
- **Dynamic/Uncertain Physics:** Martian wind storms alter terrain friction, and wheel slip can cause the rover to end up in a different coordinate than commanded, invalidating deterministic assumptions.

3.3 Building and Updating the Internal Model

- The rover utilizes an **Occupancy Grid Map** combined with **Simultaneous Localization and Mapping (SLAM)**.
- **Update Loop:**
 1. **Perceive:** Sensors capture depth and elevation data.
 2. **Update:** The rover updates probability cells in its spatial map (e.g., marking a cell as a 95% probable hazard due to slope angle).
 3. **Re-evaluate:** Cost weights on graph edges are dynamically adjusted; discovered sandy terrain is assigned a higher traversal cost than solid bedrock.

3.4 Online Search vs. Uninformed & Informed Strategies

Strategy	Operational Principle	Performance in Martian Terrain
Uninformed Search (e.g., BFS/DFS)	Explores without domain knowledge or heuristic cost estimation.	Unviable: Wastes battery power exploring irrelevant directions and risks stumbling into hazards blindly.
<i>Informed Offline Search (e.g., A)*</i>	Relies on a pre-computed static map and distance heuristic.	Fails: Cannot adapt when pre-mapped paths contain uncharted craters or obstacles discovered mid-traverse.
<i>Online Search (e.g., LRTA)*</i>	Updates heuristic estimates based on real-time sensory experience.	Optimal: Learns from dead ends, updates local cost maps, and safely guides exploration step-by-step.

3.5 Recommendation and Justification

- **Recommended Approach:** *Learning Real-Time A (LRTA)* or *D (Dynamic A)*.
- **Justification:**
 - **Safety & Adaptability:** LRTA* updates the heuristic estimate $H(s)$ of visited states to prevent the rover from looping or re-entering hazardous dead-ends.
 - **Uncertainty Management:** It operates safely under partial observability by making optimal local moves based on current sensor boundaries while preserving energy for sample collection.

4. University Exam Timetable Generation (CSP)

4.1 Problem Formulation as a CSP

- A Constraint Satisfaction Problem (CSP) is defined by a triplet $\mathcal{P} = (X, D, C)$ where variables in X must be assigned values from domains D such that all constraints in C are satisfied.

4.2 Variables, Domains, and Constraints

- **Variables (X):** The set of all scheduled exams, $X = \{E_1, E_2, \dots, E_n\}$.
- **Domains (D):** The set of available time-slot and room pairs for each exam:

$$D_i = \{(t_j, r_k) \mid t_j \in \text{TimeSlots}, r_k \in \text{Rooms}\}$$

- **Constraints (C):**
 - **Hard Constraint 1 (No Student Overlap):** For any two exams E_i, E_j sharing enrolled students, $t(E_i) \neq t(E_j)$.
 - **Hard Constraint 2 (Room Capacity & Uniqueness):** Two exams cannot share the same room at the same time: if $t(E_i) = t(E_j)$, then $r(E_i) \neq r(E_j)$. Additionally, $\text{Students}(E_i) \leq \text{Capacity}(r(E_i))$.
 - **Hard Constraint 3 (Prerequisite/Order Scheduling):** If course A must precede course B , then $t(E_A) < t(E_B)$.

- **Hard Constraint 4 (Faculty & Accommodation):** Invigilator availability schedules must be respected, and accessible halls must be assigned to exams with differently-abled students.

4.3 Backtracking Search Mechanism

- Backtracking is a depth-first search that assigns a time/room tuple to one exam variable at a time.
- If an assignment violates any constraint in C (e.g., creating a clash for a student), the algorithm immediately terminates that branch (**pruning**) and backtracks to the previous variable to try an alternative time slot.

4.4 Efficiency via Constraint Propagation

- **Forward Checking:** Whenever an exam E_i is assigned a time-slot t , Forward Checking inspects all unassigned exams E_j connected to E_i (e.g., shared students) and removes t from their domains D_j . If any D_j becomes empty, the algorithm backtracks immediately without wasting time on deeper assignments.
- **Arc Consistency (AC-3):** Enforces consistency between all pairs of variables before and during search, stripping away domain values that can never be part of a valid solution, thereby drastically shrinking the branching factor.

4.5 Real-World Challenges and Scalability Strategy

- **Real-World Challenges:** Highly dense student enrollment graphs create tight bottlenecks where standard backtracking experiences exponential runtime growth ($O(d^n)$).
- **Recommended Strategy: Backtracking combined with AC-3, Minimum Remaining Values (MRV), and Least Constraining Value (LCV) heuristics.**
- **Justification:**
 - **MRV Heuristic:** Selects the exam with the fewest legal time slots left (the most difficult exam to schedule) first, identifying inevitable conflicts early.
 - **LCV Heuristic:** Chooses the time slot that rules out the fewest options for neighboring exams.
 - **Scalability:** This synergy reduces backtracking search trees from exponential time complexity to solvable polynomial times even for large university databases.

5. Strategic Game AI for Real-Time Strategy (RTS) Game

5.1 Minimax Algorithm Modeling

- The Minimax algorithm models the RTS game as a recursive search tree where:
 - **MAX (The AI):** Attempts to maximize its payoff/advantage score.
 - **MIN (The Human Player):** Attempts to minimize the AI's score (assuming the opponent plays optimally to maximize their own advantage).
- The algorithm traverses the tree to determine the optimal move by propagating scores backward from leaf nodes to the root:

$$\text{Minimax}(s) = \begin{cases} \text{Eval}(s) & \text{if } s \text{ is a terminal/cutoff state} \\ \max_a \text{Minimax}(\text{Result}(s, a)) & \text{if it is MAX's turn} \\ \min_a \text{Minimax}(\text{Result}(s, a)) & \text{if it is MIN's turn} \end{cases}$$

5.2 Computational Challenges of Large Game Trees

- **High Branching Factor (b):** Unlike chess ($b \approx 35$), RTS games allow simultaneous unit movements, building constructions, and resource management, leading to a branching factor $b > 1000$.
- **Exponential Complexity:** Exploring a tree to depth d requires evaluating $O(b^d)$ nodes. With strict real-time frame limits (e.g., 16 ms per frame), traversing even $d = 3$ without optimization is computationally impossible.

5.3 Alpha-Beta Pruning Efficiency & Reasoning

- Alpha-Beta pruning avoids evaluating game tree branches that cannot influence the final root decision, maintaining two values:
 - α : The best (highest) score MAX is guaranteed so far.
 - β : The best (lowest) score MIN is guaranteed so far.
- **Pruning Reasoning:** While evaluating MIN's options, if a child node yields a score $v \leq \alpha$, MIN will pick this (or an even lower) value. Since MAX already has access to a move worth α elsewhere, MAX will never choose the branch leading to this MIN node. Therefore, exploration of remaining children under that node is immediately pruned.
- **Efficiency:** With optimal move ordering, Alpha-Beta pruning reduces time complexity from $O(b^d)$ to $O(b^{d/2})$, allowing the AI to search twice as deep within the same real-time window.

5.4 Evaluation Function Design

- Because reaching a terminal game-over state within the time limit is impossible, search is cut off at depth d and evaluated using a weighted linear combination heuristic:

$$\text{Eval}(s) = w_1(\Delta\text{Resources}) + w_2(\Delta\text{MilitaryPower}) + w_3(\text{MapControl}) + w_4(\text{TechTreeLevel})$$

- **Justification of Factors:**
 - $\Delta\text{Resources}$ (**Gold/Wood/Energy**): Economic superiority dictates long-term production sustainability.
 - $\Delta\text{MilitaryPower}$: Evaluates immediate combat readiness and unit health differentials.
 - **MapControl**: Quantifies strategic territory ownership, choke-point dominance, and vision radius.
 - **TechTreeLevel**: Rewards technological progression that unlocks advanced combat units.

5.5 Recommendation for Real-Time Performance

- **Recommended Strategy: Heuristic Alpha-Beta Pruning with Iterative Deepening and Quiescence Search, or Monte Carlo Tree Search (MCTS)** for macroeconomic decisions.
- **Justification:**
 - **Iterative Deepening:** Ensures that if the strict real-time timer expires mid-search, the AI immediately executes the best move calculated from the last fully completed depth level.
 - **Quiescence Search:** Prevents the "horizon effect" by continuing to search past the cutoff depth d only during volatile combat exchanges, ensuring the AI does not miscalculate trade-offs during active engagements.

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50